

Contents

1	README	1
2	02-first-login	3
3	03-tui-navigation	4
4	04-roles-and-permissions	5
5	05-logs-and-debugging	6
6	07-lobbies-and-rooms	7
7	08-client-signals	8
8	14-incident-playbooks	9
9	16-training-moderators	10
10	glossary	12

1 README

SekaiLink Core Access

SekaiLink Core Access est le cockpit d'administration de SekaiLink. Il est pense comme un outil TUI Rust/Ratatui lance depuis le bastion Nexus apres une connexion SSH, puis un login SekaiLink/Nexus.

Le nom Core Access evite la confusion avec Client Core et les libretro cores de Sekaiemu.

Objectifs

- Donner aux Admins et au role Service un environnement de travail unique.
- Centraliser logs, lobbies, rooms, releases, packs CDN, bots, scheduler, maintenance, notes et audit.
- Garder Nexus comme source d'identite, de droits, d'audit et de documentation operationnelle.
- Former les nouveaux modérateurs avec des guides imprimables et une reference de commandes complete.

Table des matieres

- [Concepts](01-concepts.md)
- [Premier login](02-first-login.md)
- [Navigation TUI](03-tui-navigation.md)
- [Roles et permissions](04-roles-and-permissions.md)
- [Logs et debugging](05-logs-and-debugging.md)
- [Users et configs](06-users-and-configs.md)
- [Lobbies et rooms](07-lobbies-and-rooms.md)
- [Signaux clients](08-client-signals.md)
- [Maintenance mode](09-maintenance-mode.md)
- [Releases et CDN](10-releases-and-cdn.md)
- [Scheduler](11-scheduler.md)
- [Bots Discord/Twitch](12-bots-discord-twitch.md)
- [Cleanup et backups](13-cleanup-and-backups.md)
- [Playbooks incidents](14-incident-playbooks.md)
- [Reference commandes](15-command-reference.md)
- [Formation modérateurs](16-training-moderators.md)
- [Runbook Admin](17-admin-runbook.md)

- [Glossaire](glossary.md)
- [Change control](change-control/README.md)

Regles fortes

- Aucun secret reel dans les docs, PDF ou exemples.
- Toute modification future doit etre documentee dans le meme commit que le code.
- Aucune modification SKLMI sans accord explicite ecrit de Jade.
- Les changements de connectivite Client Core, Sekaiemu ou SKLMI exigent un contrat documente.
- Les actions dangereuses demandent une cible tapee, une raison et un audit.

Generation PDF

Les PDF imprimables sont produits dans `dist/pdf/`:

- `sekailink-core-access-full.pdf`
- `sekailink-core-access-service-training.pdf`
- `sekailink-core-access-command-reference.pdf`
- `sekailink-core-access-incident-playbooks.pdf`
- `sekailink-core-access-quick-reference.pdf`

Commande prevue:

```
```sh
bash docs/sekailink-core-access/scripts/build-pdf.sh
```
```

La pipeline utilise Pandoc avec XeLaTeX. Le script signale clairement les dependances manquantes.

Executable MVP

Le premier binaire local vit dans `tools/core-access/`. Il fournit un shell TUI/CLI bastion avec registre de commandes, autocompletion, historique, notes, approvals locales, audit JSONL et dashboard local. Il ne modifie aucun service de production tant que l'integration Nexus n'est pas implementee.

Commandes de base:

```
```sh
cargo run --manifest-path tools/core-access/Cargo.toml
cargo run --manifest-path tools/core-access/Cargo.toml -- --command "server status all"
```
```

2 02-first-login

02 - Premier login

Preconditions

- Avoir un compte Linux autorise sur le bastion Nexus.
- Avoir un compte SekaiLink avec role Admin ou Service.
- Avoir un mapping Linux user -> SekaiLink user_id dans Nexus.

Connexion

```
```sh
ssh nexus-vps
sekailink-core-access
```
```

Core Access affiche ensuite un prompt de login SekaiLink.

Verification apres login

Executer:

```
```text
auth whoami
```
```

Resultat attendu:

- Linux user courant.
- SekaiLink user_id.
- Role.
- Capabilities.
- Heure d'expiration de session.

Premier controle

Executer:

```
```text
server status all
release current
maintenance status
```
```

Si un serveur est en rouge, ouvrir ses logs avec F8 ou:

```
```text
server logs <server> <service> --follow
```
```

3 03-tui-navigation

03 - Navigation TUI

Layout par défaut

```
```text
SekaiLink Core Access
Nexus OK | Link WARN | Worlds OK | Evolution OK | Pulse DOWN
Matrix Live filtered logs Alerts/Jobs
SERVER CPU RAM ERR 22:41 room 38293 joined approvals: 2
Link 38 64 12 22:42 SKLMI reconnect pack check: 14m
... ...

skl> room logs pas-pour-certo --follow

...```
```

## Command line

La ligne de commande est disponible partout.

Fonctions requises:

- autocomplete contextuelle;
- selection de suggestion avec Tab;
- fleches gauche/droite;
- correction au milieu de ligne;
- historique avec fleche haut/bas;
- Ctrl+A/E;
- Alt+B/F;
- Ctrl+W;
- reverse search;
- edition multi-line JSON.

## Tabs, splits et workspaces

- F11 change ou sauvegarde le layout.
- Un workspace debug capture les panels ouverts, filtres, pins et notes.
- Les workspaces sont lies a un incident, lobby, room, user ou release.

## Selection de logs

1. Placer le curseur sur le panel log.
2. Selectionner une plage.
3. F4 pour creer une note.
4. F7 pour exporter.

## 4 04-roles-and-permissions

### # 04 - Roles et permissions

#### ## Admin

Admin a le controle complet, avec confirmations et audit:

- reboot/update services;
- publish/rollback release;
- modifier users et roles;
- modifier configs utilisateur;
- give item;
- maintenance full;
- secrets set/rotate;
- approval queue.

#### ## Service

Service est quasi-admin live, mais les actions dangereuses demandent approval.

Service peut:

- lire logs, users, configs, lobbies, rooms;
- moderer lobby/chat;
- demander diagnostics client;
- request SKLMI reconnect;
- preparer un give item;
- preparer un rollback;
- preparer une fermeture de room/lobby sensible;
- poster annonces Discord/Twitch selon policy.

Service ne peut pas directement:

- reboot/update serveur;
- publier/rollback release;
- delete user;
- modifier roles;
- modifier secrets;
- modifier DB critique;
- give item sans approval.

#### ## Confirmation forte

Toute action dangereuse exige:

- cible tapee exactement;
- raison;
- resume d'impact;
- ops commit.

Exemple:

```
```text
Type target to confirm: link
Reason: emergency restart after stuck release-latest deploy
```
```

## 5 05-logs-and-debugging

# 05 - Logs et debugging

## Sources

Core Access doit lister deux vues:

- par serveur/service;
- par incident/correlation.

Sources prioritaires:

- journalctl systemd;
- fichiers logs app;
- nginx/CDN;
- Nexus auth/admin;
- Link chat/lobby;
- Worlds generation;
- Evolution packs/releases;
- Pulse;
- room multiserver logs;
- room events;
- DB/backup logs;
- client reports.

## Commandes de base

```
```text
logs list
logs list --by-server
logs list --by-incident
logs tail <source>
logs search <query>
logs filter user:<id> lobby:<id> room:<id> correlation:<id>
logs pin
logs note
logs export --format md
```
```

## Notes liees

Une note liee capture:

- extrait original;
- source;
- timestamps;
- tags;
- cible;
- ops\_commit\_id;
- auteur.

## Verbose filtre

Le cockpit doit etre verbose, mais filtre par default:

- erreurs et warnings visibles en haut;
- details disponibles en drilldown;
- bruit groupable par service, room, user ou correlation.

## 6 07-lobbies-and-rooms

### # 07 - Lobbies et rooms

#### ## Lobbies

Commandes:

```
```text
lobby list
lobby open <lobby>
lobby create
lobby edit <lobby>
lobby close <lobby>
lobby lock <lobby>
lobby unlock <lobby>
lobby chat <lobby>
lobby join-secret <lobby>
lobby broadcast <lobby> <message>
```
```

`lobby join-secret` est invisible pour les joueurs ordinaires, mais jamais pour l'audit. Il doit apparaitre comme `admin\_observer`.

#### ## Rooms

Commandes:

```
```text
room list
room open <room>
room summary <room>
room events <room>
room logs <room> --follow
room snapshot <room>
room sync <room>
room pending-items <room>
room client-reports <room>
room request-sklmi-reconnect <room> <player>
room disconnect-runtime <room> <player>
room give-item <room> <target> <item>
```
```

#### ## Give item

Policy:

- Admin ou approval Admin.
- Snapshot automatique avant action.
- Raison obligatoire.
- Impact live affiche.
- Ops commit.

#### ## SKLMI reconnect

`room request-sklmi-reconnect` envoie un signal client/runtime. Une modification SKLMI pour supporter une nouvelle forme de reconnect demande accord explicite.

## 7 08-client-signals

### # 08 - Signaux clients

Les signaux clients sont realtime. Le fallback poll peut exister pour robustesse, mais le cockpit doit viser l'immediat.

#### ## Signaux

```
```text
client broadcast
client maintenance-banner
client force-update-prompt
client request-relogin
client refresh-lobby
client refresh-room
client request-sklmi-reconnect
client restart-runtime
client clear-cache-request
client diagnostics-request
```
```

#### ## Diagnostics

Regles:

- consentement utilisateur obligatoire;
- logs et configs redacted;
- upload lie a un incident ou ops commit;
- jamais de secret client brut.

#### ## Broadcast

Ciblage:

- global;
- server;
- lobby;
- room;
- role;
- version;
- game.

Exemple:

```
```text
broadcast lobby pas-pour-certo "Maintenance de room dans 5 minutes."
```
```

## 8 14-incident-playbooks

### # 14 - Playbooks incidents

#### ## Room server down

1. `room list`
2. `room summary <room>`
3. `room logs <room> --follow`
4. `server status link`
5. `logs filter room:<room>`
6. F4 pour noter les lignes importantes.
7. Si necessaire: F12 -> snapshot rooms.

#### ## SKLMI non connecte

1. `room summary <room>`
2. Verifier presence player et runtime.
3. `room events <room>`
4. `client request-sklmi-reconnect <player>`
5. Si echec: demander diagnostics avec consentement.

#### ## Client update brise

1. `release current`
2. `release verify-cdn`
3. `logs filter release:<version>`
4. `maintenance enable client\_update\_only --message "...`
5. `release rollback <version>` si Admin.

#### ## Generation bloquee

1. `server status worlds`
2. `logs tail worlds:generation`
3. `schedule history health-probe-worlds`
4. `maintenance enable generation\_only --message "...`

#### ## CDN pack invalid

1. `pack repo list`
2. `pack validate <id>`
3. `pack rollback <id> <version>`
4. `broadcast game <game> "...`

#### ## Bot Discord/Twitch down

1. `discord status` ou `twitch status`
2. `discord logs` ou `twitch logs`
3. `discord reload` ou `twitch connect <channel>`
4. Creer note incident si recurring.

#### ## Maintenance d'urgence

1. F12 Panic.
2. Lire impact.
3. Choisir scope minimal.
4. Taper cible + raison.
5. Broadcast.
6. Collect logs bundle.

## 9 16-training-moderators

# 16 - Formation modérateurs

## Objectif

Former le role Service pour aider en live sans mettre la production en danger.

## Parcours

1. Lire Concepts.
2. Lire Premier login.
3. Lire Navigation TUI.
4. Lire Roles et permissions.
5. Faire les exercices logs.
6. Faire les exercices lobbies/rooms.
7. Faire un incident simule.
8. Lire les limites Service.

## Exercices

### Exercice 1 - Diagnostiquer un joueur

```
```text
user search <name>
user open <user>
user sessions <user>
user configs <user>
```
```

But: trouver si le joueur est connecte, sur quel device et avec quelle config.

### Exercice 2 - Suivre une room

```
```text
room summary <room>
room events <room>
room logs <room> --follow
```
```

But: identifier qui est connecte, quel runtime parle, et s'il y a des pending items.

### Exercice 3 - Creer une note

1. Ouvrir un log.
2. Selectionner une plage.
3. F4.
4. Ajouter tags.
5. Exporter en Markdown.

## A ne jamais faire sans Admin

- publier une release;
- rollback release;
- reboot/update serveur;
- modifier role;
- modifier secret;
- injecter item;
- modifier DB critique;
- ignorer un backup gate.

## Checklist avant premier shift

- Je sais me connecter au bastion.
- Je sais verifier mon role.
- Je sais lister logs et rooms.
- Je sais creer une note.
- Je sais demander approval.
- Je sais utiliser F12 Panic en lecture avant action.

## 10 glossary

# Glossaire

**\*\*Admin\*\***: role complet avec actions dangereuses, approvals et maintenance.

**\*\*Approval queue\*\***: file de demandes preparees par Service et approuvees par Admin.

**\*\*Bastion Nexus\*\***: serveur Nexus utilise comme point d'entree SSH et hote de Core Access.

**\*\*Client Core\*\***: app desktop SekaiLink utilisateur.

**\*\*Core Access\*\***: cockpit TUI admin. Ne pas confondre avec Client Core ou les libretro cores.

**\*\*Evolution\*\***: role distribution, CDN, assets, packs et releases.

**\*\*Link\*\***: API publique, lobbies, chat, room passthrough, release-latest.

**\*\*Nexus\*\***: identite, users, configs, audit, scheduler et source de confiance.

**\*\*Ops commit\*\***: entree atomique d'audit pour une action.

**\*\*Pulse\*\***: assistant/config helper.

**\*\*Room\*\***: session runtime/live associee a une sync.

**\*\*Sekaieму\*\***: emulator renderer/runtime visuel.

**\*\*Service\*\***: role support/moderation puissant, avec approvals pour actions dangereuses.

**\*\*SKLMI\*\***: companion tracker/runtime qui possede la logique tracker.

**\*\*Worlds\*\***: generation service.